

MBS Matched Betting System

Frontend & Origin Hardening Report

Date: 2026-02-15 | Scope: Next.js Frontend deployment on Oracle Cloud VPS with Cloudflare DNS/Zero Trust

This document captures the complete, end-to-end hardening journey performed on the MBS frontend deployment and its origin infrastructure. It starts from an Nginx reverse-proxy setup (Cloudflare Origin Certificate + security headers) and ends with a full Zero Trust architecture based on Cloudflare Tunnel for web traffic and Cloudflare Access for SSH, eliminating inbound exposure of ports 80/443/22 to the public Internet.

1. Executive Summary

- Frontend: Next.js running in Docker and exposed internally on 127.0.0.1:3002.
- Backend: Node.js container exposed internally on 127.0.0.1:3000 (unchanged by frontend hardening).
- Initial edge/origin model: Cloudflare (proxy) -> public origin IP -> Nginx (80/443) -> containers.
- Intermediate hardening: canonical non-www, HTTP->HTTPS redirects, strict security headers, HSTS, gzip, static caching.
- mTLS (Authenticated Origin Pulls): Nginx required Cloudflare client cert; initial failure fixed by using Cloudflare Origin Pull CA certificate.
- Final architecture (Zero Trust): Cloudflare -> Cloudflare Tunnel (outbound-only) -> localhost containers. No inbound 80/443.
- SSH moved to Zero Trust: Cloudflare Access protected SSH hostname; SSH reachable via ProxyCommand with cloudflared. No inbound 22 required.

2. Target State Architecture

Before (traditional origin): Cloudflare (proxy) → Public IP (VPS) → Nginx (80/443) → Frontend (localhost:3002) / Backend (localhost:3000).

After (Zero Trust total): Cloudflare Edge → Cloudflare Tunnel (outbound from VPS) → Frontend (127.0.0.1:3002) and Backend (127.0.0.1:3000). SSH via Cloudflare Access → Tunnel → localhost:22.

Component	Before	After (Zero Trust)
Web exposure	Inbound 80/443 open (even if Cloudflare-only)	No inbound 80/443; origin not reachable on web ports
TLS termination	Cloudflare + Nginx origin cert/mTLS	Cloudflare edge; tunnel provides secure transport
Origin identity	Authenticated Origin Pulls (mTLS)	Tunnel authentication; origin isolated
SSH exposure	22 whitelisted (risk when IP changes)	No public 22; SSH via Cloudflare Access + tunnel

3. Detailed Step-by-Step Hardening Timeline

This section lists every meaningful step executed during the hardening process, including the exact commands and configuration changes.

3.1 Nginx Frontend Virtual Host: Canonical non-www + HTTPS

Goal: expose the frontend under matched-betting-system.com with canonical redirects (www → non-www) and enforce HTTPS.

File edited:

```
/etc/nginx/conf.d/frontend.conf

# Redirect HTTP www → HTTPS non-www
server {
    listen 80;
    server_name www.matched-betting-system.com;
    return 301 https://matched-betting-system.com$request_uri;
}

# Redirect HTTP non-www → HTTPS non-www
server {
    listen 80;
    server_name matched-betting-system.com;
    return 301 https://matched-betting-system.com$request_uri;
}

# Redirect HTTPS www → HTTPS non-www
server {
    listen 443 ssl http2;
    server_name www.matched-betting-system.com;

    ssl_certificate /etc/nginx/ssl/origin.crt;
    ssl_certificate_key /etc/nginx/ssl/origin.key;

    return 301 https://matched-betting-system.com$request_uri;
}

# HTTPS Canonical (non-www)
server {
    listen 443 ssl http2;
    server_name matched-betting-system.com;

    ssl_certificate /etc/nginx/ssl/origin.crt;
    ssl_certificate_key /etc/nginx/ssl/origin.key;

    # Security headers (baseline)
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;
    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;

    server_tokens off;

    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml application/xml;
    gzip_min_length 256;

    location / {
        proxy_pass http://127.0.0.1:3002;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

```

        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }

    location ~* \.(js|css|png|jpg|jpeg|gif|svg|ico|woff2)$ {
        expires 30d;
        access_log off;
        add_header Cache-Control "public, max-age=2592000, immutable";
    }
}

```

Validation commands:

```

sudo nginx -t
sudo systemctl reload nginx

```

3.2 Authenticated Origin Pulls (mTLS) on Nginx

Goal: ensure the origin accepts HTTPS traffic only if Cloudflare presents a valid client certificate (mTLS).

Key configuration (applied ONLY on the canonical HTTPS server block):

```

ssl_client_certificate /etc/nginx/cloudflare/cloudflare.crt;
ssl_verify_client on;

```

Critical fix: the CA certificate used for verification must be the Cloudflare Origin Pull CA certificate (not the domain Origin Certificate). When the wrong certificate was used, browsers reported SSL handshake errors.

Correct CA retrieval (static Cloudflare URL), saved as:

```

/etc/nginx/cloudflare/cloudflare.crt

sudo mkdir -p /etc/nginx/cloudflare
sudo curl -o /etc/nginx/cloudflare/cloudflare.crt \
  https://developers.cloudflare.com/ssl/static/authenticated_origin_pull_ca.pem
sudo nginx -t
sudo systemctl reload nginx

```

3.3 Next.js Build and Dockerization (Frontend)

Goal: build the Next.js app in production mode and run it in Docker behind the reverse proxy (initial phase).

Observed build issues and resolutions:

- TypeScript error: 'page.tsx is not a module' (resolved by ensuring pages export a default component and by fixing project path/alias issues).
- Turbopack route conflict: parallel route groups (dashboard/public) resolving to the same path; fixed by adjusting folder structure so paths are unique.
- Module alias error: cannot find module '@src/config/env' (resolved by aligning tsconfig paths and imports).

Result: local build succeeded; Docker image built successfully and frontend responded on localhost:3002.

3.4 Migration to Zero Trust: Cloudflare Tunnel for Web

Goal: remove origin exposure by routing all HTTP(S) through Cloudflare Tunnel (outbound-only) directly to localhost services.

Install cloudflared on Oracle Linux (ARM) using the binary (dpkg is not available on Oracle Linux):

```
sudo curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-  
-o /usr/local/bin/cloudflared  
sudo chmod +x /usr/local/bin/cloudflared  
cloudflared --version
```

Authenticate to Cloudflare and create tunnel:

```
cloudflared tunnel login  
cloudflared tunnel create mbs-tunnel  
cloudflared tunnel list
```

Configure tunnel routing (direct-to-containers):

```
/etc/cloudflared/config.yml  
  
tunnel: 87c5c5c9-6173-439c-b0fe-daad5a8de8a3  
credentials-file: /etc/cloudflared/87c5c5c9-6173-439c-b0fe-daad5a8de8a3.json  
  
ingress:  
- hostname: matched-betting-system.com  
  service: http://127.0.0.1:3002  
- hostname: www.matched-betting-system.com  
  service: http://127.0.0.1:3002  
- hostname: api.matched-betting-system.com  
  service: http://127.0.0.1:3000  
- service: http_status:404
```

DNS routing encountered an error because A/CNAME records already existed (Cloudflare error 1003). Resolution: remove conflicting A/CNAME records for root/api/www and re-run the route commands to create tunnel records.

```
cloudflared tunnel route dns mbs-tunnel matched-betting-system.com  
cloudflared tunnel route dns mbs-tunnel www.matched-betting-system.com  
cloudflared tunnel route dns mbs-tunnel api.matched-betting-system.com
```

3.5 Running cloudflared as a systemd Service

Goal: keep the tunnel stable via systemd with restart-on-failure.

Issue: `cloudflared service install --config ...` was not supported by the installed version. Resolution: install service without flags and use /etc/cloudflared/config.yml.

```
sudo /usr/local/bin/cloudflared service install  
sudo systemctl enable --now cloudflared  
sudo systemctl status cloudflared --no-pager
```

Service failure root-cause: credentials file path referenced a placeholder and/or a user-home location not accessible by the service. Log showed:

```
Tunnel credentials file '/home/opc/.cloudflared/<UUID>.json' doesn't exist or is not a file
```

Fix: move tunnel credentials JSON to /etc/cloudflared and update config.yml to the exact filename.

```
# Example (use your UUID filename)  
sudo mv /home/opc/.cloudflared/87c5c5c9-6173-439c-b0fe-daad5a8de8a3.json /etc/cloudflared/  
sudo chown root:root /etc/cloudflared/87c5c5c9-6173-439c-b0fe-daad5a8de8a3.json  
sudo chmod 600 /etc/cloudflared/87c5c5c9-6173-439c-b0fe-daad5a8de8a3.json  
  
sudo nano /etc/cloudflared/config.yml  
# set:  
# credentials-file: /etc/cloudflared/87c5c5c9-6173-439c-b0fe-daad5a8de8a3.json  
  
sudo systemctl restart cloudflared  
sudo systemctl status cloudflared --no-pager
```

3.6 Remove Inbound Web Exposure (OCI Security List)

Goal: after tunnel verification, remove all inbound TCP 80/443 rules so the origin cannot be reached directly via its public IP.

Validation before cut:

```
curl -I http://127.0.0.1:3000
curl -I http://127.0.0.1:3002
```

Then, in OCI Security List: remove ALL ingress rules for TCP 80 and TCP 443 (both IPv4 and IPv6 if present).

After removal: direct requests to https:// must fail, while domain requests must continue to work via tunnel.

3.7 Cleanup: Disable Nginx (Not Needed with Tunnel)

Goal: reduce moving parts and remove unused TLS/mTLS configs once tunnel is the only ingress.

```
sudo systemctl stop nginx
sudo systemctl disable nginx
sudo ss -tulnp | grep -E "80|443"
```

3.8 Zero Trust SSH: Cloudflare Access + Tunnel

Goal: regain SSH access from any network while keeping port 22 closed to the Internet. Access is protected by Cloudflare Access policy (OTP/email) and tunnel transport.

Tunnel config: add SSH hostname mapping:

```
- hostname: ssh.matched-betting-system.com
  service: ssh://localhost:22
```

Windows client setup:

```
# C:\Users\emanu\.ssh\config
Host mbs-ssh
  HostName ssh.matched-betting-system.com
  User opc
  ProxyCommand cloudflared access ssh --hostname %h

ssh mbs-ssh
```

3.9 Close Inbound SSH (OCI Security List)

Goal: once SSH via Access is confirmed, remove any public ingress rule for TCP 22.

4. Validation Checklist

- Tunnel service is running: `sudo systemctl status cloudflared` shows active (running).
- Web domains resolve and load over HTTPS: matched-betting-system.com and api.matched-betting-system.com.
- Direct origin IP access fails for HTTPS: https:// unreachable.
- No inbound web ports in OCI Security Lists: no 80/443 ingress rules.
- SSH via Access works from any network: `ssh mbs-ssh` succeeds.
- Direct SSH to origin IP fails: `ssh opc@` times out/refused.
- Optional: Nginx disabled and not listening on 80/443.

Useful verification commands on the VPS:

```
sudo ss -tulnp
sudo systemctl status cloudflared --no-pager
journalctl -u cloudflared -n 50 --no-pager
curl -I http://127.0.0.1:3002
curl -I http://127.0.0.1:3000
```

5. Rollback and Recovery Notes

- Re-open OCI ingress rules for 80/443 and re-enable Nginx if you want the traditional origin model back.
- Re-open OCI ingress rule for 22 (whitelist only your IP) if Cloudflare Access is misconfigured and you need emergency SSH.
- If tunnel DNS records were created, you may need to restore A/CNAME records pointing to the origin IP to revert DNS.
- Keep at least one emergency access path: OCI Console/Serial Console or a temporary SSH whitelist rule with strict timeboxing.

6. Recommended Next Steps (Enterprise Observability & IDS)

- Monitoring: external uptime checks (e.g., Cloudflare Health Checks or third-party) for frontend and API endpoints.
- Logging: centralize logs (cloudflared, docker, application) with retention (e.g., Loki/Promtail, ELK, or a managed log platform).
- Host auditing: enable auditd rules for SSH, sudo, and sensitive file changes.
- File Integrity Monitoring: AIDE baseline + scheduled checks.
- Container hardening: read-only filesystem where feasible, drop capabilities, and pin images by digest.
- Backup strategy: configuration + secrets + DB backups with offsite storage and tested restore procedure.